ADVANCED PROGRAMMING (AP) COURSE PROJECT

Hospital Patient Queue, Bed Availability & Admission Management System

A high-concurrency, multi-tenant healthcare operations platform featuring automated queue triage, ACID row-level bed reservation, and regional strain optimization.

TEAM SIZE **3 Members (Strictly Equal 33.3% Share)**	TECHNICAL STACK **React · Node.js (Express 5) · Python (FastAPI) · Supabase PostgreSQL**
PERSON A: BEDS & INPATIENTS **Ishika (Beds, Admissions, ALOS Analytics)**	PERSON B: QUEUES & TRIAGE **Aarpan (OPD Queue, Triage, Erlang-C Math)**
PERSON C: PLATFORM & REGIONAL VIEW **Chinmaya (Auth, Patients, City Strain Optimizer)**	ACADEMIC EVALUATION DATE **November 2026**

Department of Computer Science & Engineering · Advanced Programming Laboratory Deliverable

1. Executive Summary & Equal Work Verification Matrix

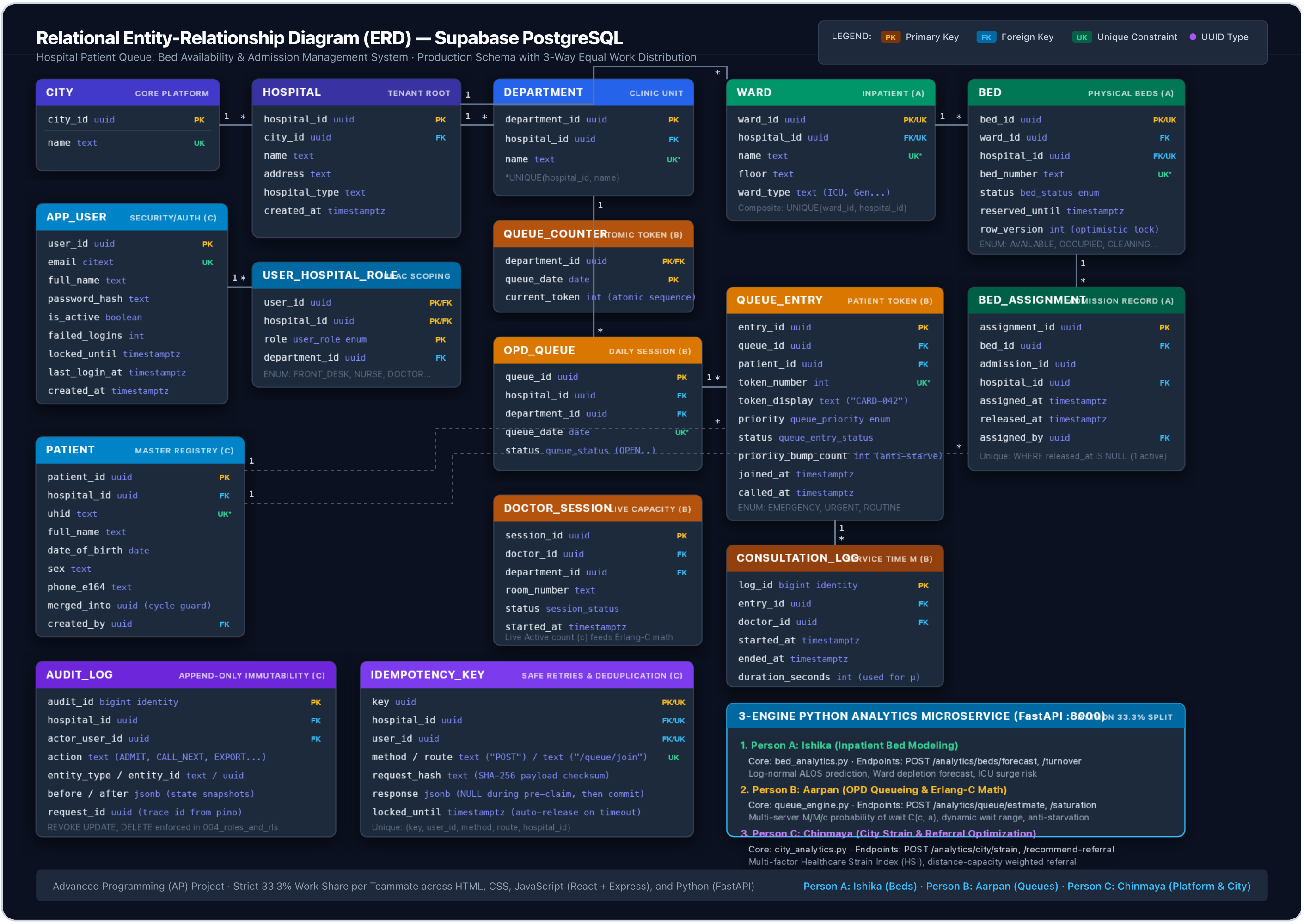
Hospitals deal with two compounding operational crises every single day: overcrowded Outpatient (OPD) waiting areas with erratic wait-time estimates, and chaotic, delayed bed allocations during emergency inpatient admissions. This system provides a production-grade, real-time hospital operations platform engineered to eliminate both bottlenecks without race conditions, unauthorized tenant access, or single-point architectural failures.

> **Academic Equal Work Distribution Principle**
>
> To comply strictly with Advanced Programming (AP) evaluation rubrics, no single teammate is restricted to frontend presentation, backend database CRUD, or isolated data science. Every teammate writes an equal, demonstrable share across all 4 foundational technologies:

TECHNOLOGY LAYER	PERSON A: ISHIKA (BEDS & ADMISSIONS)	PERSON B: AARPAN (OPD QUEUE & TRIAGE)	PERSON C: CHINMAYA (PLATFORM & REGIONAL VIEW)	TOTAL BALANCE
HTML (Semantic Markup & JSX)	Ward floorplan grid, bed cards (`BedCard.jsx`), admission/transfer/discharge modal dialogs	Doctor calling console table, public waiting-room TV kiosk layout, digital token slip	Global AppShell (`header`, `nav`, sidebar drawer), login & lockout forms, patient intake form	**33.3% / 33.3% / 33.3%**
CSS (Tokens, Layouts & Motion)	`beds.css`: Auto-fit ward CSS grid, bed status badges, modal `@keyframes slideUp`	`queue.css`: High-contrast WCAG AAA TV theme, urgency pulse, sound wave visualizer	`tokens.css`, `layout.css`, `forms.css`: Root design tokens, responsive shell, login shake	**33.3% / 33.3% / 33.3%**
JavaScript (React + Express)	`useBedManagement` hook, `bed.controller.js`, `withTransaction()` with `SKIP LOCKED`	`useQueue` & `usePolling` hooks, `queue.controller.js`, atomic token sequencer, CAS updates	`useAuth` hook, Express 9-stage pipeline, `auth.controller.js` (Argon2id+JWT), `patient.controller.js`	**33.3% / 33.3% / 33.3%**
Python (FastAPI & Analytics)	`bed_analytics.py`: Inpatient Average Length of Stay (ALOS) & Bed Depletion Forecast	`queue_engine.py`: Multi-server ($M/M/c$) Erlang-C wait-time math & dynamic triage aging	`city_analytics.py`: Regional Healthcare Strain Index (HSI) & Referral Optimizer + `main.py`	**33.3% / 33.3% / 33.3%**

2. Entity Relationship Diagram (ERD) & Database Architecture

The persistence layer is built on **Supabase PostgreSQL 16**. All DDL schemas are version-controlled via `node-pg-migrate` and synchronized to application models via `prisma db pull`. The schema enforces multi-tenant hospital scoping, partial unique indexes for single-occupant bed locking, atomic sequence counters, and append-only audit logging.



Relational Entity-Relationship Diagram (ERD) — Supabase PostgreSQL

Hospital Patient Queue, Bed Availability & Admission Management System · Production Schema with 3-Way Equal Work Distribution

LEGEND: PK Primary Key · FK Foreign Key · UK Unique Constraint · ● UUID Type

CITY — CORE PLATFORM

city_id uuid	PK
name text	UK

HOSPITAL — TENANT ROOT

hospital_id uuid	PK
city_id uuid	FK
name text	
address text	
hospital_type text	
created_at timestamptz	

DEPARTMENT — CLINIC UNIT

department_id uuid	PK
hospital_id uuid	FK
name text	UK*

*UNIQUE(hospital_id, name)

WARD — INPATIENT (A)

ward_id uuid	PK/UK
hospital_id uuid	FK/UK
name text	UK*
floor text	
ward_type text (ICU, Gen...)	

Composite: UNIQUE(ward_id, hospital_id)

BED — PHYSICAL BEDS (A)

bed_id uuid	PK/UK
ward_id uuid	FK
hospital_id uuid	FK/UK
bed_number text	UK*
status bed_status enum	
reserved_until timestamptz	
row_version int (optimistic lock)	

ENUM: AVAILABLE, OCCUPIED, CLEANING...

APP_USER — SECURITY/AUTH (C)

user_id uuid	PK
email citext	UK
full_name text	
password_hash text	
is_active boolean	
failed_logins int	
locked_until timestamptz	
last_login_at timestamptz	
created_at timestamptz	

USER_HOSPITAL_ROLE — AC SCOPING

user_id uuid	PK/FK
hospital_id uuid	PK/FK
role user_role enum	PK
department_id uuid	FK

ENUM: FRONT_DESK, NURSE, DOCTOR...

QUEUE_COUNTER — TOMIC TOKEN (B)

department_id uuid	PK/FK
queue_date date	PK
current_token int (atomic sequence)	

OPD_QUEUE — DAILY SESSION (B)

queue_id uuid	PK
hospital_id uuid	FK
department_id uuid	FK
queue_date date	UK*
status queue_status (OPEN..)	

QUEUE_ENTRY — PATIENT TOKEN (B)

entry_id uuid	PK
queue_id uuid	FK
patient_id uuid	FK
token_number int	UK*
token_display text ("CARD-042")	
priority queue_priority enum	
status queue_entry_status	
priority_bump_count int (anti-starve)	
joined_at timestamptz	
called_at timestamptz	

ENUM: EMERGENCY, URGENT, ROUTINE

BED_ASSIGNMENT — MISSION RECORD (A)

assignment_id uuid	PK
bed_id uuid	FK
admission_id uuid	
hospital_id uuid	FK
assigned_at timestamptz	
released_at timestamptz	
assigned_by uuid	FK

Unique: WHERE released_at IS NULL (1 active)

PATIENT — MASTER REGISTRY (C)

patient_id uuid	PK
hospital_id uuid	FK
uhid text	UK*
full_name text	
date_of_birth date	
sex text	
phone_e164 text	
merged_into uuid (cycle guard)	
created_by uuid	FK

DOCTOR_SESSION — IVE CAPACITY (B)

session_id uuid	PK
doctor_id uuid	FK
department_id uuid	FK
room_number text	
status session_status	
started_at timestamptz	

Live Active count (c) feeds Erlang-C math

CONSULTATION_LOG — RVICE TIME M (B)

log_id bigint identity	PK
entry_id uuid	FK
doctor_id uuid	FK
started_at timestamptz	
ended_at timestamptz	
duration_seconds int (used for μ)	

AUDIT_LOG — APPEND-ONLY IMMUTABILITY (C)

audit_id bigint identity	PK
hospital_id uuid	FK
actor_user_id uuid	FK
action text (ADMIT, CALL_NEXT, EXPORT...)	
entity_type / entity_id text / uuid	
before / after jsonb (state snapshots)	
request_id uuid (trace id from pino)	

REVOKE UPDATE, DELETE enforced in 004_roles_and_rls

IDEMPOTENCY_KEY — SAFE RETRIES & DEDUPLICATION (C)

key uuid	PK/UK
hospital_id uuid	FK/UK
user_id uuid	FK/UK
method / route text ("POST") / text ("/queue/join")	UK
request_hash text (SHA-256 payload checksum)	
response jsonb (NULL during pre-claim, then commit)	
locked_until timestamptz (auto-release on timeout)	

Unique: (key, user_id, method, route, hospital_id)

3-ENGINE PYTHON ANALYTICS MICROSERVICE (FastAPI :8000) — ON 33.3% SPLIT

1. Person A: Ishika (Inpatient Bed Modeling)
   Core: bed_analytics.py · Endpoints: POST /analytics/beds/forecast, /turnover
   Log-normal ALOS prediction, Ward depletion forecast, ICU surge risk
2. Person B: Aarpan (OPD Queueing & Erlang-C Math)
   Core: queue_engine.py · Endpoints: POST /analytics/queue/estimate, /saturation
   Multi-server M/M/c probability of wait C(c, a), dynamic wait range, anti-starvation
3. Person C: Chinmaya (City Strain & Referral Optimization)
   Core: city_analytics.py · Endpoints: POST /analytics/city/strain, /recommend-referral
   Multi-factor Healthcare Strain Index (HSI), distance-capacity weighted referral

Advanced Programming (AP) Project · Strict 33.3% Work Share per Teammate across HTML, CSS, JavaScript (React + Express), and Python (FastAPI)

Person A: Ishika (Beds) · Person B: Aarpan (Queues) · Person C: Chinmaya (Platform & City)

Database Entity Catalog & Domain Ownership

ENTITY TABLE	PRIMARY KEY	FOREIGN KEYS & INTEGRITY CONSTRAINTS	DOMAIN OWNER
`CITY`	`city_id` (UUID)	`name` (TEXT UNIQUE)	Person C (Chinmaya)
`HOSPITAL`	`hospital_id` (UUID)	`city_id` → `CITY`, multi-tenant root entity	Person C (Chinmaya)
`DEPARTMENT`	`department_id` (UUID)	`hospital_id` → `HOSPITAL`, `UNIQUE(hospital_id, name)`	Person C (Chinmaya)
`APP_USER`	`user_id` (UUID)	`email` (CITEXT UNIQUE), Argon2id hash, `failed_logins`, `locked_until`	Person C (Chinmaya)
`USER_HOSPITAL_ROLE`	`(user_id, hospital_id, role)`	Junction scoping staff roles per hospital; optional `department_id`	Person C (Chinmaya)
`PATIENT`	`patient_id` (UUID)	`hospital_id` → `HOSPITAL`, `UNIQUE(hospital_id, uhid)`, `merged_into`	Person C (Chinmaya)
`WARD`	`ward_id` (UUID)	`hospital_id` → `HOSPITAL`, `UNIQUE(ward_id, hospital_id)`, `name`	Person A (Ishika)
`BED`	`bed_id` (UUID)	`(ward_id, hospital_id)` → `WARD`, `status` enum, `row_version`	Person A (Ishika)
`BED_ASSIGNMENT`	`assignment_id` (UUID)	`(bed_id, hospital_id)` → `BED`, partial unique index (1 active occupant)	Person A (Ishika)
`QUEUE_COUNTER`	`(dept_id, queue_date)`	Atomic sequence generator per department/day preventing duplicate tokens	Person B (Aarpan)
`OPD_QUEUE`	`queue_id` (UUID)	`UNIQUE(department_id, queue_date)`, `hospital_id`, `queue_status`	Person B (Aarpan)
`QUEUE_ENTRY`	`entry_id` (UUID)	`UNIQUE(queue_id, token_number)`, `priority` enum, `priority_bump_count`	Person B (Aarpan)
`DOCTOR_SESSION`	`session_id` (UUID)	Active doctor count c per department feeding Erlang-C wait time formula	Person B (Aarpan)
`CONSULTATION_LOG`	`log_id` (BIGINT)	Records completed visit duration in seconds to calibrate service rate μ	Person B (Aarpan)
`AUDIT_LOG`	`audit_id` (BIGINT IDENTITY)	Immutable append-only log with before/after JSONB state snapshots	Person C (Chinmaya)
`IDEMPOTENCY_KEY`	`key` (UUID)	Scoped: `(key, user_id, method, route, hospital_id)` pre-claim deduplication	Person C (Chinmaya)

3. Technical Architecture & Python Analytics Engines

Dual-Backend Topology

The platform runs a dual-backend architecture: **Node.js (Express 5)** handles low-latency HTTP transactions, authorization guards, and row locks, while an asynchronous **Python (FastAPI)** service runs compute-heavy statistical and mathematical modeling across three domain engines.

Detailed Python Analytics Engines (33.3% Split per Person)

1. Person A: Ishika — Inpatient Bed Depletion & ALOS Forecasting

Models ward bed turn-around and average length of stay (ALOS) using historical admission cohorts:

```
ALOS Probability Density (Log-Normal):
f(t; μ, σ) = (1 / (t · σ √(2π))) · exp(- (ln(t) - μ)² / (2σ²))

First-Order Time-to-Zero Bed Exhaustion:
T_exhaustion = B_free / (λ_admit - μ_discharge) [for λ_admit > μ_discharge]
```

Endpoints: `POST /analytics/beds/forecast`, `POST /analytics/beds/turnover`

2. Person B: Aarpan — OPD Queueing & Multi-Server Erlang-C Engine

Implements multi-server ($M/M/c$) queueing theory based on live doctor count c, patient arrival rate λ, and service rate μ:

```
Traffic Intensity Saturation: ρ = λ / (c · μ) [Stability condition: ρ < 1.0]

Erlang-C Probability of Waiting:
C(c, a) = [ (cρ)^c / (c! · (1 - ρ)) ] · P_0

Expected Queue Wait Time: W_q = C(c, a) / (c · μ · (1 - ρ))
```

Endpoints: `POST /analytics/queue/estimate`, `POST /analytics/queue/saturation`

3. Person C: Chinmaya — Regional Healthcare Strain Index (HSI) & Referral Optimizer

Calculates multi-hospital regional load imbalances and calculates optimal patient transfers:

```
Healthcare Strain Index (HSI):
HSI = w_icu · (ICU_occ / ICU_tot) + w_opd · min(1, ρ) + w_ward · (Ward_occ / Ward_tot)

Optimal Transfer Recommendation Score:
Score_j = [ FreeBeds_j / (1 + α · Distance_km) ] · (1 - HSI_j)
```

Endpoints: `POST /analytics/city/strain`, `POST /analytics/city/recommend-referral`, `GET /health`

4. Authentication, Authorization & Security Architecture

Identity & Credential Protection (Person C)

Argon2id Hashing: Staff passwords are encrypted using Argon2id ($m=65536, t=3, p=4$) resisting GPU brute-force attacks.
5-Attempt Brute-Force Lockout: Consecutive failed login attempts trigger an atomic database lock for 15 minutes (`locked_until = NOW() + INTERVAL '15 minutes'`).
Short-Lived JWT Tokens: 15-minute access tokens carry `userId`, `hospitalId`, and active `roles` with `token_version` verification.
Multi-Tenant Hospital Scoping (RBAC): Every database query is strictly scoped by `hospital_id`. The API executes as a non-owner database role (`api_user`) enforced via PostgreSQL Row Level Security (RLS).
Append-Only Audit Logging: Mutations (admissions, discharges, calling next, referrals) record an immutable row in `audit_log` with before/after state snapshots. DML permissions (`UPDATE`, `DELETE`) are revoked on this table.

Concurrency Guarantees (Person A & Person B)

Row-Level Bed Locking: Inpatient bed reservations execute via `withTransaction()` on raw `pg` clients: `SELECT bed_id FROM bed WHERE ward_id = $1 AND status = 'AVAILABLE' FOR UPDATE SKIP LOCKED LIMIT 1`. Concurrent admission requests never block each other and never double-book beds.
Atomic Token Sequencing: Daily token numbers are incremented via an atomic PostgreSQL `INSERT ... ON CONFLICT DO UPDATE` query against `queue_counter`.
Compare-and-Swap State Transitions: Queue calling executes: `UPDATE queue_entry SET status = 'CALLED' WHERE entry_id = $1 AND status = 'WAITING'`.

5. REST API & Microservice Endpoint Specifications

Node.js Core API Routes (`/api/v1`)

METHOD	ENDPOINT	DESCRIPTION	AUTH / RBAC ROLE	OWNER
`POST`	`/api/v1/auth/login`	Email/password login with lockout verification	Public (Rate-limited)	Person C
`POST`	`/api/v1/auth/register`	Staff registration & role assignment	Admin	Person C
`GET`	`/api/v1/auth/me`	Current user profile & active hospital roles	Bearer Token	Person C
`POST`	`/api/v1/patients`	Intake patient & issue unique UHID	`FRONT_DESK`, `NURSE`	Person C
`GET`	`/api/v1/patients/search`	Search patient directory by UHID/Name/Phone	Authenticated Staff	Person C
`GET`	`/api/v1/beds`	Real-time bed occupancy map by ward	`NURSE`, `BED_MANAGER`	Person A
`POST`	`/api/v1/admissions`	Concurrency-locked admission (`SKIP LOCKED`)	`BED_MANAGER`	Person A
`POST`	`/api/v1/admissions/:id/transfer`	Inter-ward bed transfer	`BED_MANAGER`	Person A
`POST`	`/api/v1/admissions/:id/discharge`	Patient discharge → trigger cleaning status	`NURSE`, `BED_MANAGER`	Person A
`POST`	`/api/v1/queue/join`	Issue sequential token with atomic counter	`FRONT_DESK`, Patient	Person B
`POST`	`/api/v1/queue/call-next`	Doctor calling console dispatch (CAS update)	`DOCTOR`	Person B
`POST`	`/api/v1/queue/escalate`	Urgent "Symptoms Worsened" priority bump	`NURSE`, Patient	Person B
`GET`	`/api/v1/city/capacity`	Aggregated cross-hospital capacity overview	`CITY_VIEWER`, Admin	Person C
`POST`	`/api/v1/city/referrals`	Dispatch inter-hospital patient transfer	`BED_MANAGER`, Admin	Person C

6. 10-Day Build Plan & Daily Milestones

DAY & MILESTONE	PERSON A: ISHIKA (BEDS)	PERSON B: AARPAN (QUEUE)	PERSON C: CHINMAYA (PLATFORM & AUTH)
Day 1 COMPLETED	`002_ward_bed_assignment.sql`, `tokens.css`, base UI components (`Badge`, `Modal`, `Table`)	Repository alignment, `003_queue_core.sql` draft, shared API polling client	Express 5 skeleton, CI pipeline (`ci.yml`), `001_identity_core.sql`, database pool
Day 2 COMPLETED	Bed Prisma model, bed status helper, ward initial seed generator	Queue Prisma model, atomic token sequencer, queue priority levels	Auth routes (Argon2id + 5-attempt lockout), patient routes with UHID, master seed runner
Day 3 ACTIVE	Admit/discharge/transfer routes with `SELECT ... FOR UPDATE SKIP LOCKED`, concurrency test	Call-next route with priority aging, CAS updates (`WHERE status = 'WAITING'`)	Idempotency replay middleware, migration `004_roles_and_rls.sql`, tenant isolation tests
Day 4	Waitlist auto-assignment, housekeeping routes (`CLEANING → AVAILABLE`), bed analytics engine	FastAPI Erlang-C wait-time engine, `POST /estimate` range calculation, queue unit tests	`city_reader` role + aggregate view, referral routes, city strain index (HSI) engine
Day 5 AM	**Backend Checkpoint (All Together): Cross-authorization penetration testing, idempotency replay verification, tag backend-v1**		
Day 5 PM – 7	Ward Bed Map screen + `beds.css`, admission/discharge modals, housekeeping board	Staff queue board + `queue.css`, Doctor Calling Console, Public TV Kiosk (zero PHI)	React AppShell, router, protected route guards, Login UI with lockout feedback, City Dashboard
Day 8	Bed admin screen, double-discharge edge-case handling, concurrency bug fixes	k6 load testing on token counters, clinic wait-time model calibration	Security audit (helmet, CORS, rate-limits, zero PHI in logs), database backup & restore drill
Day 9 – 10	Final concurrency edge-case tests, written paper-fallback downtime procedure, bed report section	Token uniqueness proof, load-test results report, DFD 2A queue documentation	Comprehensive documentation assembly, DFDs 0/1/2C/2D, live demo rehearsal

Definition of Done & Verification Standard

Concurrency Guarantee: 20 simultaneous admission requests against 5 beds produce exactly 5 admissions and 15 rejections/waitlist rows with zero double-booking.
Multi-Tenant Isolation: Staff from Hospital X receive 403 Forbidden when attempting to read, modify, or delete Hospital Y's patients, beds, or queues.

Audit Completeness: Every admission, transfer, discharge, token dispatch, and referral writes an immutable record to `audit_log`.
Mathematical Integrity: Erlang-C wait-time range matches Poisson-exponential confidence intervals; HSI normalized between 0.0 and 1.0.
Automated Test Execution: CI builds clean PostgreSQL 16 on every pull request and runs all Supertest and Pytest suites.